malloc 研究与创意案例

故障现场黑匣子的动态快照设计

课程方向 嵌入式 Linux 整理日期 2026 年 10 月 21 日

1 研究目标与背景

本报告梳理 malloc 的背景、演进、分配机制和正确用法，并实现一个“故障现场黑匣子”：日常采样使用固定环形缓冲区，异常触发时才申请独立快照，使历史现场能够在后续覆盖采样中继续保留。该案例通过具体数据生命周期解释动态分配的作用。

本例研究普通用户态的动态内存管理，使用标准 C11 和模拟输入，不依赖板卡外设。实际验证在 Windows x64 完成，Linux 与鲁班猫运行尚待验证。

1 1 动态内存解决的问题

普通自动局部对象通常在离开作用域后结束生命周期，定长数组还需要预先选择容量。当数据大小或保留时间由运行时事件决定时，动态分配可以让程序独立控制对象的大小与生命周期。程序同时承担释放、失败处理和越界检查责任。[1]

1 2 可核对的演进节点

节点	实现变化
UNIX V7 历史源码	已有 malloc、free 等接口，使用循环首次适配并在搜索时合并空闲块。[3]
1987 年	Doug Lea 开始编写后来称为 dlmalloc 的分配器。[4]
2001 年版本节点	ptmalloc2 在 dlmalloc 基础上适配多线程和多个 arena，glibc 源码头部保留这一线索。[5]
glibc 2.26	2017 年的该版本文档已记录线程缓存 tcache 相关参数。[6]

上述节点说明实现演进，不用于断言 malloc 的准确发明年份。C 标准约束接口语义，glibc 只是具体实现之一；历史文章和旧版本参数不能直接代表当前所有 Linux 系统。

2 Linux 中的分配机制

2 1 应用接口与库实现

malloc 是 C 库函数。以 glibc 为例，分配器处理对齐和块管理信息，优先尝试复用自己已经管理的空闲空间。tcache 是线程缓存，bins 组织不同类别的空闲块，arena 管理相关空间及状态。缓存命中时，一次 malloc 可以完全在用户态完成。[2][5]

库需要更多空间时，可能通过 brk 或 mmap 与内核交互。具体路线受库版本、参数和已有空间影响，不能记成“每次 malloc 都调用内核”，也不能用固定大小阈值解释所有系统。[2][6]

2 2 虚拟地址和物理内存

应用获得的是可供对象使用的连续虚拟地址范围，物理页未必连续。Linux 的页面建立还可能延迟到首次访问；启用乐观分配时，返回非 NULL 也不能作为未来一定不会出现内存压力的保证。用户态地址不能直接当成设备需要的物理连续 DMA 缓冲区。[2]

2 3 释放和碎片

free 使分配空间能够再次使用。分配器可能将空间保留在缓存或 arena 中，所以释放后进程 RSS 不一定立即降低。内存管理还要承担元数据与对齐开销。[2][4]

已有区域示意	块 A	空闲	块 B	空闲	块 C
大小	16 B	16 B	16 B	16 B	16 B
状态	使用中	可复用	使用中	可复用	使用中

此模型空闲总量为 32 B，但没有一段可容纳 24 B 的空闲块，体现外部碎片。真实分配器仍可能扩展区域或建立映射，因此不能仅据此判定 malloc 必然失败。大小取整带来的块内浪费属于另一类开销。[4]

2 4 嵌入式应用的影响

资源有限的设备需要限制总量、处理失败并明确所有者。通用堆分配的锁、搜索及页面访问等路径不提供普遍的硬实时时延保证。严格采样时序通常需要预分配缓冲区或快照池，并将较慢的处理放在后台任务。

3 接口用法与常见错误

接口	用途	使用要点
malloc(bytes)	申请字节数	内容未初始化，先检查 NULL。
calloc(n, size)	数组分配与清零	清零的是各字节，不应推广为所有类型的语义零值。
realloc(p, bytes)	改变分配大小	可能搬迁，扩大部分未初始化，非零申请失败时原块仍有效。
free(p)	释放分配对象	传入有效分配起始地址或 NULL，不重复释放。

接口声明位于 <stdlib.h>。下面片段可放入返回 int 的函数中，SIZE_MAX 需要 <stdint.h>。sizeof *p 跟随元素类型变化，避免把指针大小误当成整个数组大小。[1]

```
size_t n = 10;
int *p = NULL;
if (n == 0 || n > SIZE_MAX / sizeof *p)
    return EXIT_FAILURE;
p = malloc(n * sizeof *p);
if (p == NULL) return EXIT_FAILURE;
for (size_t i = 0; i < n; ++i) p[i] = 0;
free(p);
p = NULL;
```

下面是独立的扩容片段，p 应指向仍有效的动态分配空间。realloc 先用临时指针接收结果，成功后只使用返回指针；直接覆盖唯一旧指针会在失败时失去原块地址。[1]

```
/* new_bytes > 0，且已完成大小检查 */
int *tmp = realloc(p, new_bytes);
if (tmp == NULL) { /* p 仍有效，按原所有权最终释放 */ }
else { p = tmp; }
```

常见错误包括乘法上溢、读取未初始化数据、写出数组边界、忘记释放、重复释放和释放后访问。把一个变量设为 NULL 只能改变该变量，其他别名仍可能悬空。本案例避免零大小分配及其跨标准差异。

4 创意案例筛选

选题以“malloc 是否解决具体问题、与嵌入式场景是否相关、是否容易复现实验”为标准。以下是本次设计取舍，不是全网题目数量统计或独创性证明。

候选方案	能说明的重点	取舍
学生信息链表	节点增删和逐块释放	已有同类教育项目可检索，场景与嵌入式联系较弱。[8]
传感器数组动态扩容	容量变化和 realloc	便于入门，但主线仍是通用动态数组。
手写简化分配器	空闲链表和块切分	容易把作业转为实现分配器，增加错误和解释负担。
故障现场黑匣子	独立生命周期、变长快照、预算及失败隔离	选用。普通采样与事件快照采用不同存储策略。

4 1 选定问题

设备只保留最近几帧实时数据时，旧内容会不断被覆盖。如果异常发生后仍然只保存一个指向实时缓冲区的指针，所谓“故障现场”很快就会变化。因此，每次异常需要保留一份由事件独立拥有的内容副本。malloc 为该副本提供独立空间与保留时间。

4 2 具体辨识度

方案将固定容量采样与事件触发分配组合起来。事件发生时有多少上下文就保存多少，启动早期事件只复制已有帧；每份快照用一次 malloc 同时容纳头部与帧数据；预算不足或申请失败时拒绝当前事件，历史快照保持完整。实验专门验证后续覆盖不能篡改已保存现场。

4 3 与已有方法的关系

环形缓冲区与快照并非本作业首创。Linux ftrace 已有保留某时刻缓冲区的快照机制，其实现可以交换当前缓冲区与备用缓冲区。[7] 本例是围绕 malloc 编写的简化用户态程序，使用独立复制，不调用或复制 ftrace 实现，也不宣称发明了黑匣子概念。

5 黑匣子结构与内存管理

5 1 两种数据生命周期

实时环形缓冲区只保留最近 W 帧，装满后覆盖最旧帧。事件快照在触发时复制已有内容，之后直到统一释放才结束生命周期。默认 W=4，快照包含触发帧及最多 3 帧历史，不包含未来帧。

```
typedef struct {
    unsigned int sequence;
    int temperature_tenths;
} Sample;

typedef struct {
    size_t event;
    size_t count;
    Sample frames[];
} Snapshot;
```

frames[] 是柔性数组成员，实际数据与头部位于同一次申请中。需要的字节数按目标平台 sizeof 计算，先检查上溢，再检查预算，最后申请和初始化。

```
if (count > (SIZE_MAX - sizeof(Snapshot)) / sizeof(Sample))
    return;
size_t bytes = sizeof(Snapshot) + count * sizeof(Sample);
if (bytes > budget - used) return;
Snapshot *s = malloc(bytes);
if (s == NULL) return;
```

上述为关键判断的简化片段，省略参数、日志和计数；完整可编译实现见 blackbox.c 的 capture 函数。全部字段复制完成后，记录器才登记快照并增加 used。预算检查维持 0 ≤ used ≤ budget，该不变量由初始值、登记与统一释放共同保证。

5 2 所有权和资源边界

记录器拥有所有成功申请的快照，report 只读取，release_all 逐块 free 并清空指针。一次快照只需要一次申请和一次释放；固定指针表最多保存 8 份快照。拒绝新事件不会释放或覆盖旧快照，申请失败也不会中断后续模拟采样。

budget 仅约束快照请求字节数之和，不含分配器元数据、固定缓冲区、栈和库占用，不能用它声称整个进程只使用同样多的内存。

6 实验输入与结果

本次实际环境：Windows x64，TinyCC 0.9.27；编译参数为 -std=c11 -Wall -Werror。样本和快照头部大小分别为 8 B、16 B。以下是该环境实测，完整输出保存在代码目录 run_*.txt；不作为 Linux 或 ARM 板卡记录。

模拟温度序列为 25、42、27、28、29、46、26、27、49、26、51、25 °C。每一帧达到 40 °C 都独立触发事件，事件帧为 2、6、9、11。没有引入迟滞或去抖。

触发帧	快照保存的序号	帧数	申请大小
2	1、2	2	32 B
6	3、4、5、6	4	48 B
9	6、7、8、9	4	48 B
11	8、9、10、11	4	48 B

6 1 全部事件成功保存

```
./blackbox --budget 256
LIVE_RING final_frames=9,10,11,12
SNAPSHOT event=2 frames=1:25.0,2:42.0
SUMMARY events=4 saved=4 rejected=0 alloc_failed=0 used=176
CLEANUP blocks=4 released=176 remaining=0
```

输出为关键行摘录。最终实时环形缓冲区只剩第 9 至 12 帧，第 2 帧事件仍保留第 1、2 帧，直接说明快照内容独立于实时缓冲区。退出状态为 0。

6 2 预算不足时拒绝当前事件

```
./blackbox --budget 128
REJECT event=11 reason=budget_or_slots bytes=48 used=128
SUMMARY events=4 saved=3 rejected=1 alloc_failed=0 used=128
CLEANUP blocks=3 released=128 remaining=0
```

前三份快照的请求总量恰好为 128 B，第 11 帧事件无法再申请 48 B，程序拒绝该事件并保留此前三份。退出状态为非零，用于显式报告有事件未保存。其他平台字节数以 CONFIG 行为准。

7 失败隔离与复现方法

7 1 可重复的分配失败实验

```
./blackbox --budget 256 --fail-after 1
CAPTURE event=2 frames=2 bytes=32 used=32
ALLOC_FAIL event=6 previous_snapshots_preserved
ALLOC_FAIL event=9 previous_snapshots_preserved
ALLOC_FAIL event=11 previous_snapshots_preserved
SUMMARY events=4 saved=1 rejected=0 alloc_failed=3 used=32
CLEANUP blocks=1 released=32 remaining=0
```

--fail-after 1 允许首份快照申请成功，此后包装函数模拟返回 NULL。已有第 2 帧快照仍完整，实时采样继续到第 12 帧，最后正常释放已保存空间。该实验没有耗尽机器内存，不检验真实 OOM 或内核 overcommit。

7 2 已完成的检查

31 项自动行为检查全部通过，覆盖窗口 1 与 16、启动早期事件、环形覆盖后的快照不变、预算恰好够用与差一个字节、全部或部分申请失败、快照数量上限、无事件输入和非法参数。测试校验实际帧序号与温度，而不只检查一条“成功”输出。

7 3 Linux 或鲁班猫复现

执行位置为普通 Linux 终端或已登录鲁班猫的 SSH 终端。将 blackbox_demo 复制到个人目录后进入该目录运行，无需 sudo。若没有 cc 但有 gcc，可替换编译器名。编译失败时停止并记录第一条错误。

```
cc -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror \
   blackbox.c -o blackbox
./blackbox --budget 256
python3 test_blackbox.py ./blackbox
```

Makefile 另外提供 make test 和可选 make sanitize。当前未执行 Linux、ARM 或 ASan/UBSan 检查，不能据本机行为检查断言所有内存问题已被排除。

7 4 实现限制

数据为模拟输入，快照仅存在于进程 RAM，不具备断电保留能力。采样、申请、复制和输出在同一线程中进行，不保证硬实时。真实严格时序任务应评估后台处理和预分配快照池；采样阈值也必须依据具体设备重新设计。

8 结论与参考资料

malloc 提供运行时对象分配能力，其实际效果取决于应用所有权设计和 C 库实现。黑匣子案例将短期循环数据与需要独立保留的事件现场分开，用单块变长快照解释申请大小、对象生命周期和释放责任，并通过预算拒绝与故障注入验证失败路径。

本次已完成资料研究、案例筛选、完整代码及本机验证。案例不依赖鲁班猫外设，但板卡运行尚未验证；是否另需上板材料，应以课程额外要求为准。

[1] WG14 N1570 C11 委员会草案，7.22.3 内存管理函数

https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[2] Linux man-pages malloc 3

https://man7.org/linux/man-pages/man3/malloc.3.html

[3] UNIX V7 malloc 历史源码，TUHS 存档

https://www.tuhs.org/cgi-bin/utree.pl?file=V7/usr/src/libc/gen/malloc.c

[4] Doug Lea A Memory Allocator

https://gee.cs.oswego.edu/dl/html/malloc.html

[5] glibc malloc.c 源码镜像，历史注释与实现

https://github.com/bminor/glibc/blob/master/malloc/malloc.c

[6] glibc 2.26 Memory Allocation Tunables

https://sourceware.org/glibc/manual/2.26/html_node/Memory-Allocation-Tunables.html

[7] Linux 5.16 ftrace 文档 Snapshot 一节

https://www.kernel.org/doc/html/v5.16/trace/ftrace.html#snapshot

[8] 学生信息链表示例，用于候选选题对照

https://github.com/asserhossam74/student-management-system-c

资料核对日期为 2026 年 10 月 21 日。标准草案用于接口语义，历史材料用于历史节点，指定版本文档只支持该版本描述。快照设计与源代码为本作业的具体实现，不以搜索结果少来证明题目唯一。